# Cloakroom: A Peer-to-Peer Dark Pool for Shielded Zcash

Chris Gachau

October 2026

**Abstract.** A purely private version of exchange would allow shielded Zcash to be traded for other assets without ever leaving the shielded pool. Today every route out of shielded ZEC passes through a transparent address controlled by a custodian, a bridge, or an intent network, which publishes amounts and timing and can hold funds. Atomic swaps remove the custodian, but the hash-locked construction needs scripts that Zcash's shielded pools do not have, and it links both legs of a trade with a shared hash. We propose a swap in which the ZEC never touches a transparent address. The two parties split the spend authorizing key of a fresh Ironwood address into additive shares; the counterparty's tokens are locked on an EVM chain in a contract with two exits, each opened by revealing exactly one share. Whichever exit is taken publishes the share the other party is missing, so the trade either settles on both sides or is refunded on both sides. On Zcash the swap is two ordinary fully shielded transactions; on the EVM chain it reveals the price. A request-for-quote protocol carried in encrypted memos turns the swap into a dark pool in which the price is public and the participants are not. We implemented the protocol and ran it end to end: against the production `orchard` verifier, on a local Zebra node, in seven two-party scenarios (five of them adversarial), and in two swaps between Zcash testnet and Base Sepolia, in which every Zcash transaction was fully shielded.

## 1. Introduction

Zcash [1, 2] gives its users two kinds of money. Transparent value behaves like Bitcoin [3]: every address, amount and transfer is public. Shielded value lives in a pool of encrypted notes, and a fully shielded transaction shows an observer only that it happened and what fee it paid. Most shielded ZEC now sits in the Ironwood pool, which the NU6.3 upgrade introduced in July 2026 [4].

Shielded value stays private only while it stays shielded. Each crossing between the transparent and shielded pools publishes the amount moved and when it moved, and matching crossings by amount and timing has linked a large share of shielded activity to transparent addresses [8, 9, 10]; the same methods recently linked 17.65% of withdrawals from Railgun on Ethereum [11]. Exchange is where the crossings happen. Major exchanges withdraw ZEC only to transparent addresses. NEAR Intents, the most used swap route, supports ZEC on transparent addresses only [12], and Maya and THORChain hold ZEC in transparent vaults. Every route also adds a custodian: in 2026 one user's $589,000 USDT payout from a ZEC swap was held by NEAR Intents in a compliance review for more than fifty days [13], and on October 1 NEAR Intents was exploited for about $3.8 million and paused [14].

Atomic swaps [15, 16] remove the custodian, but the usual construction locks both legs to the same hash preimage behind a timelock. Zcash's shielded pools have no script system, so the lock has to sit on transparent outputs, where it publishes the amount, and the shared hash then appears on both chains and links the two legs [17]. Monero, which also lacks scripts, solved this differently: the Monero leg is sent to an address whose spending key is split between the parties, and the other leg is built so that taking it reveals the missing share [18, 20].

We apply that idea to the Orchard protocol, which the Ironwood pool uses, and build a market on top of it. Our contributions are:

- A swap between shielded ZEC and any EVM token whose Zcash leg is two ordinary fully shielded transactions and never uses a transparent address (Sections 4 and 5).
- An EVM escrow that checks a revealed scalar against a stored Pallas point directly, so no cross-curve equality proof is needed (Section 4.3).
- An analysis that names what each observer learns, including three pitfalls that are easy to miss: the shared outgoing viewing key, the fact that a reverted transaction still publishes its secret, and the light-wallet server learning which transactions are ours (Sections 5 to 7).
- A request-for-quote protocol in 512-byte encrypted memos (Section 8).
- An implementation, and evidence at four levels: the production verifier, a local Zebra node, seven two-party

scenarios, and two swaps on public test networks (Sections 9 and 10).

## 2. Background

### 2.1 Orchard keys

The Orchard protocol [2, §4.2.3] works over the Pallas curve $E_p : y^2 = x^3 + 5$, whose group has prime order $q$. A spending key yields a *spend authorizing key* $\mathsf{ask} \in \mathbb{F}_q^*$, a *nullifier deriving key* $\mathsf{nk} \in \mathbb{F}_p$, and a *commitment randomness* $\mathsf{rivk} \in \mathbb{F}_q$. The *spend validating key* is $\mathsf{ak} = [\mathsf{ask}]\,\mathcal{G}$, where $\mathcal{G}$ is the RedPallas spend-authorization base point $\mathsf{GroupHash}(\texttt{"z.cash:Orchard"}, \texttt{"G"})$. A full viewing key encodes $\mathsf{ak}$ by its $x$-coordinate alone [2, §5.6.4.4], so key generation fixes the sign of its $y$-coordinate: if $\tilde{y}(\mathsf{ak}) = 1$, $\mathsf{ask}$ is negated. The triple $(\mathsf{ak}, \mathsf{nk}, \mathsf{rivk})$ is the full viewing key. From it come the incoming viewing key $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{rivk}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$, which must not be 0 or $\perp$; an outgoing viewing key $\mathsf{ovk}$, derived with $\mathsf{rivk}$ as a PRF key; and an unbounded family of mutually unlinkable *diversified addresses* $(d, \mathsf{pk}_d = [\mathsf{ivk}]\,g_d)$.

### 2.2 Spending

A shielded transaction is a set of *actions*, each pairing one spend with one output [2]. To spend a note, the spender picks a fresh randomizer $\alpha$, publishes the randomized key $\mathsf{rk} = \mathsf{ak} + [\alpha]\mathcal{G}$, and signs the transaction's ZIP 244 sighash [28] with RedPallas under $\mathsf{rsk} = \mathsf{ask} + \alpha$. A Halo 2 proof [29] shows, without saying which note is spent, that $\mathsf{rk}$ randomizes the $\mathsf{ak}$ behind that note's address, that the note is in the commitment tree, and that the published nullifier is correct. Because $\alpha$ is uniform, $\mathsf{rk}$ is a uniform point that reveals nothing about $\mathsf{ak}$. An action publishes only a value commitment, a nullifier, $\mathsf{rk}$, a note commitment, an ephemeral key, two ciphertexts and a signature, and the bundle adds a proof and a binding signature.

### 2.3 Partially created transactions

A partially created Zcash transaction (PCZT) [30] lets different machines construct, prove and sign a transaction. A Signer receives each action's $\alpha$ and the sighash; the `orchard` crate accepts an externally produced signature only if it verifies under $\mathsf{rk}$. Threshold signing for Zcash, FROST [31, 32], uses this interface to spend from a key that no single machine holds.

### 2.4 Ironwood and NU7

In May 2026 a soundness flaw was found in the Orchard circuit. NU6.2 deployed a fixed circuit, and NU6.3 ("Ironwood", July 28, 2026) introduced a new pool that uses the fixed circuit, and sealed the old Orchard pool so value can only leave it [4]. Ironwood reuses the Orchard key structure and address format, and one incoming viewing key scans both pools [5]. ZIP 2005 adds an optional key derivation, $\mathsf{use_qsk}$, that makes notes recoverable in a quantum emergency [6]. NU7 has been active on testnet since October 4, 2026 and is scheduled for mainnet; it shortens the block interval to 25 seconds and keeps the transaction format [7].

## 3. Related Work

**Hash-locked swaps.** Nolan's 2013 construction [15], formalized by Herlihy [16], and the 2017 Zcash–Bitcoin XCAT [17] need a script that checks a hash preimage and a timelock. In Zcash this exists only for transparent outputs.

**Monero swaps.** Gugger [18] showed that a chain without scripts can be swapped against Bitcoin if its leg is locked to a key split between the parties and the Bitcoin leg uses adaptor signatures to reveal a share, with two timelocks $t_0$ and $t_1$. COMIT ran the protocol in production from 2021 [19]; the project continues as eigenwallet. AthanorLabs replaced the Bitcoin leg with an Ethereum contract that checks a revealed secp256k1 scalar [20]. Because Monero keys live on Ed25519, that design also needs a proof that one scalar underlies points on both curves [21]. Our escrow checks the Pallas point directly. Its windows also differ: AthanorLabs allows a claim when ready before $t_0$ or between $t_0$ and $t_1$; we allow a claim whenever the swap is ready, up to $t_1$, which shortens the happy path and opens no race, because refunds are closed from Ready until $t_1$.

**Other designs.** Universal atomic swaps [22] need only signature verification on each chain and use adaptor signatures with time-lock puzzles; since Orchard spend authorization is a Schnorr-type signature they could in principle

apply to shielded ZEC, but we know of no RedPallas implementation. Zswap [23] is a different scheme with a similar name: multi-asset swaps inside a single shielded ledger.

**Zcash.** Zwap [24] has offered Orchard–ETH/USDC swaps capped at $100 since May 2026. Its announcement describes a script lock on the Orchard side and a hash-time-locked EVM contract bound to the note by a client-side proof. Forum reviewers objected that Orchard has no script system to enforce such a lock, and users reported the service unreachable from August. In Cloakroom nothing on the Zcash side has to be enforced by a script: the note is locked by a key that neither party holds alone. The zswap grant proposal [25] sought adaptor signatures over RedPallas and was closed without code in September 2026.

**Dark pools.** Renegade [26] matches orders at the midpoint with multi-party computation on Base and Arbitrum. Penumbra [27] runs every swap in a block as a batch at one clearing price, and plans sealed-bid encryption of inputs. Neither settles shielded ZEC.

## 4. The Protocol

### 4.1 Setting

A *taker* $T$ holds shielded ZEC and wants $u$ tokens on an EVM chain; a *maker* $M$ holds the tokens. We use USDC on Base throughout. The parties agree on a ZEC amount $v$, a sweep fee $f$, a swap identifier $\mathsf{sid}$, and two deadlines $t_0 < t_1$. Neither trusts the other.

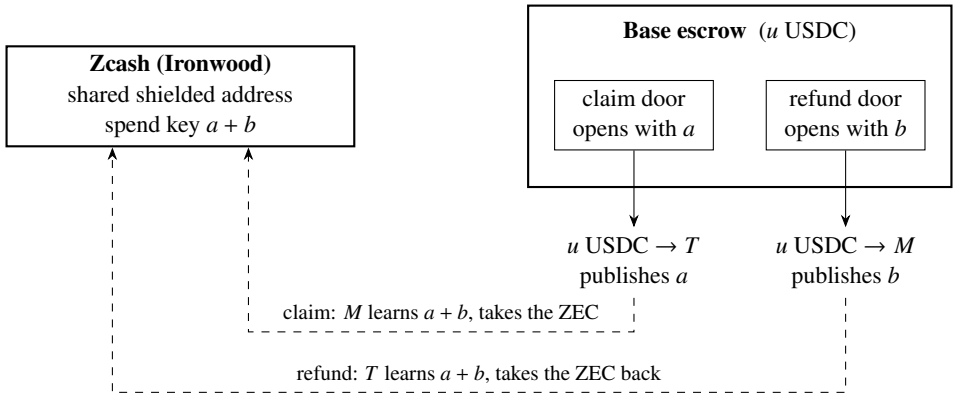



**Figure 1.** The ZEC is locked to the sum of two shares. Each exit from the escrow is opened by one share and publishes it, which completes the other party's knowledge of $a + b$.

### 4.2 Key setup

1. $T$ samples $a \leftarrow \mathbb{F}_q^*$ and a seed $s_T$; $M$ samples $b \leftarrow \mathbb{F}_q^*$ and a seed $s_M$, all fresh for this swap. They exchange $A = [a]\mathcal{G}$ and $B = [b]\mathcal{G}$ with Schnorr proofs of knowledge whose challenges bind $\mathsf{sid}$ and the party's role. Each party rejects a proof that fails, and a proof made for one role does not verify for the other.

2. $T$ commits to $s_T$ before seeing $s_M$ and opens the commitment after; both derive $\mathsf{nk} = H_{\mathbb{F}_p}(\texttt{"nk"}, \mathsf{sid}, s_T, s_M)$ and $\mathsf{rivk} = H_{\mathbb{F}_q}(\texttt{"rivk"}, \mathsf{sid}, s_T, s_M)$, where $H$ is BLAKE2b-512 reduced into the field.

3. If $\tilde{y}(A + B) = 1$, both replace $(a, A)$ by $(-a, -A)$ and $(b, B)$ by $(-b, -B)$. The rule is deterministic, so both reach the same result, and $\mathsf{ak} = A + B$ then has $\tilde{y} = 0$ as the key encoding requires. In 2,000 random trials 990 needed the negation.

4. Both form $\mathsf{FVK}_J = (\mathsf{ak}, \mathsf{nk}, \mathsf{rivk})$, check that it is a valid full viewing key (in particular $\mathsf{ivk} \notin \{0, \perp\}$), and derive the shared address $\mathsf{addr}_J$: the external-scope address at diversifier index 0, encoded as a unified address with only an Orchard receiver, which receives into Ironwood.

After setup both parties can *see* any note sent to $\mathsf{addr}_J$, and neither can *spend* it, because spending requires a signature under $\mathsf{ask}_J = a + b$.

The proofs of knowledge are essential. Without them $M$ could choose $B = [b']\mathcal{G} - A$, so that $\mathsf{ak} = [b']\mathcal{G}$ is a key he alone controls: the rogue-key attack known from Schnorr multisignatures [33].

Our derivation of $\mathsf{nk}$ and $\mathsf{rivk}$ is the $\mathsf{use_qsk} = \mathsf{false}$ branch of ZIP 2005 [6], which Ironwood accepts [5]. Notes at a joint address are therefore not quantum-recoverable. A joint note lives for minutes, so we accept this. A $\mathsf{use_qsk} = \mathsf{true}$ variant needs the parties to agree privately on a shared secret from which $\mathsf{nk}$ and the quantum keys derive, with $\mathsf{rivk} = H^{\mathsf{rivk}}_{\mathsf{qk}}(\mathsf{ak}, \mathsf{nk})$; no library implements that derivation yet.

### 4.3 Escrow contract

For each swap the escrow stores the maker, a payout address chosen by $T$, the token and amount $u$, a relayer fee, $A$ and $B$ as affine Pallas points, $t_0$, $t_1$, and a state in $\{\mathsf{Open}, \mathsf{Ready}, \mathsf{Claimed}, \mathsf{Refunded}\}$:

- `open`: called by $M$. It checks that $A$ and $B$ lie on the curve and that $now < t_0 < t_1$, takes $u$ tokens from $M$, and enters Open.
- `setReady`: called by $M$, in Open, before $t_0$.
- `claim`($a$): callable by anyone before $t_1$, in Ready, or in Open from $t_0$. It checks $[a]\mathcal{G} = A$, pays $u$ minus the relayer fee to the payout address and the fee to the caller, enters Claimed, and emits $a$.
- `refund`($b$): callable by anyone, in Open before $t_0$, or in Open or Ready from $t_1$. It checks $[b]\mathcal{G} = B$, returns $u$ to $M$, enters Refunded, and emits $b$.

Claimed and Refunded are terminal, so at most one exit ever succeeds. Payouts go to addresses fixed at `open`, so anyone may submit a share without being able to redirect funds, and a taker whose payout address holds no ETH can have a relayer submit her claim.

No EVM chain has a Pallas precompile, so the contract does the scalar multiplication itself: Jacobian coordinates, the $a = 0$ doubling and mixed-addition formulas, and a projective comparison with the stored point ($X = xZ^2$, $Y = yZ^3$), so no field inversion is needed. Checking the scalar against $A$ directly is what lets us skip the cross-curve proof that the Monero–Ethereum design requires [20, 21].

### 4.4 Settlement

1. **Lock.** $M$ calls `open`. $T$ checks every stored parameter and waits until the call is on Base's *safe* head, that is, included in a batch posted to Ethereum.
2. **Fund.** $T$ sends $v + f$ ZEC to $\mathsf{addr}_J$ in a fully shielded transaction from her own wallet.
3. **Confirm.** $M$ trial-decrypts with $\mathsf{ivk}_J$, checks the value, waits for $k_Z$ confirmations, and calls `setReady` before $t_0$. If no adequate note arrives, he calls `refund`.
4. **Claim.** $T$, or a relayer, submits `claim`($a$), and $T$'s payout address receives the tokens.
5. **Sweep.** $M$ reads $a$ from the `Claimed` event, computes $\mathsf{ask}_J = a + b$, and spends the joint note to an address of his own, signing externally with $\mathsf{ask}_J + \alpha$. The sweep uses $\mathsf{ovk} = \perp$ and returns no change to $\mathsf{addr}_J$ (Section 7).

If the swap is abandoned, $M$'s `refund`($b$) publishes $b$ and $T$ sweeps the note back to herself with $a + b$.

### 4.5 Fees

The joint note pays for its own sweep. Under ZIP 317 [34] the fee is 5,000 zatoshis per logical action with a minimum of two; for Orchard-protocol bundles the logical-action count is the number of actions, and the draft Revision 1 for NU6.3 adds the same term for Ironwood. A sweep with one spend and one output is padded to two actions, so $f = 10{,}000$ zatoshis. On Base, $M$ pays for `open` and `setReady`, and the relayer fee pays whoever submits `claim`.

## 5. Deadlines and Liveness

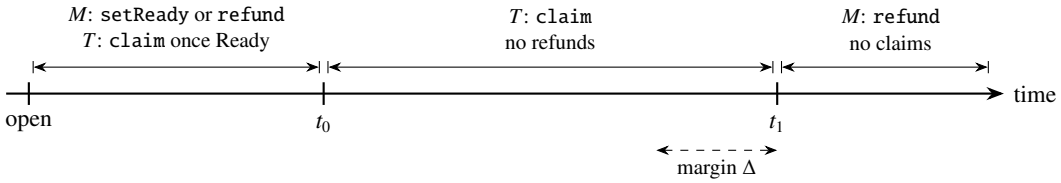



**Figure 2.** Escrow windows. A taker's wallet stops attempting claims at $t_1 - \Delta$.

Shielded Zcash cannot enforce a timelock, so all deadlines live in the escrow, and the share that each exit publishes carries the outcome to Zcash. Table 1 covers every case in which a party stops responding.

| Who stops responding | Outcome |
|---|---|
| $T$, before funding | No note arrives; $M$ refunds before $t_0$. Nothing is at stake. |
| $T$, after funding | $M$ sets Ready. At $t_1$ he refunds and publishes $b$; whenever $T$ returns she sweeps the note with $a + b$. That sweep has no deadline. |
| $M$, after `open` | From $t_0$, $T$ claims without Ready and publishes $a$; the note waits at $\mathsf{addr}_J$ until $M$ returns. |
| Both | After $t_1$ only `refund` remains, and only $M$ knows $b$. A rational $M$ refunds, since otherwise his own tokens stay locked; $T$ then recovers her ZEC. |

**Table 1.** Liveness outcomes.

The protocol therefore needs presence at two points. $M$ must act before $t_0$, which a market maker does automatically. $T$ must claim between Ready and $t_1$; if she does not, she is refunded, unless $M$ has also disappeared for good, in which case both parties' funds stay locked and $M$ loses as much as $T$. Monero swaps rely on the same incentive. Five wallet rules make the windows safe:

**R1** $T$ funds only after `open` is on Base's safe head and every stored parameter matches the quote.

**R1b** $T$ does not fund within $2\Delta$ of $t_0$, so that $M$ can still confirm the note and set Ready.

**R2** $T$ broadcasts `claim`($a$) only if Ready has been observed or $t_0 + \delta$ has passed, and only before $t_1 - \Delta$.

**R3** $M$ broadcasts `refund`($b$) only when the refund condition holds with margin $\delta$.

**R4** Shares are fresh for each swap and erased after settlement.

R2 and R3 matter because *a reverted transaction still publishes its calldata*. A claim that lands after $t_1$ fails but reveals $a$, after which $M$ can both refund and sweep; a refund that fails before $t_1$ reveals $b$, after which $T$ can both claim and sweep. A party must never broadcast a share unless the contract would accept it, with margin. Rule R1b was added after our first local run (Section 10.4).

**Censorship.** Base's sequencer can delay a transaction but cannot exclude it forever. Transactions deposited through the `OptimismPortal` contract on Ethereum must be included within the sequencing window, 3,600 Ethereum blocks or about 12 hours on Base [35, 36]; this still holds after Base's May 2026 Azul upgrade replaced its OP Stack node software. In production we set $t_1 - t_0 \geq 24$ hours so that a censored taker can force-include her claim. A relayer that withholds a claim can delay it but not redirect it, and $T$ can resubmit by another route.

**Reorganizations.** $M$ waits $k_Z$ Zcash confirmations before Ready, because if the funding note were reorganized away after Ready, $T$ could claim without having paid. $T$ waits for Base's safe head before funding, because a reorganized `open` would leave her note at an address with no escrow; she could recover it only with $M$'s cooperation. For production we suggest $t_0$ = open + 1 hour, $t_1 = t_0 + 24$ hours, and $k_Z$ equal to about 15 minutes of blocks. Our tests used shorter windows (Section 10).

## 6. Security

We assume that discrete logarithms are hard on Pallas, that RedPallas with rerandomized keys is unforgeable [2], that the proofs of knowledge are extractable, and that a party following R1–R4 gets its transactions included on Base within the sequencing window.

**Atomicity.** The escrow lets at most one of `claim` and `refund` succeed. The joint note can be spent only by someone who knows $a + b$. Before either exit, $T$ knows only $a$ and $M$ only $b$; the proofs of knowledge and the hardness of discrete logarithms stop either from learning the other's share or choosing their own as a function of it. A successful `claim` publishes $a$ and pays $T$, after which only $M$ knows $a + b$. A successful `refund` publishes $b$ and pays $M$, after which only $T$ knows $a + b$. A third party who sees one published share still lacks the other. The outcome is therefore both sides settled, both refunded, or, in the last row of Table 1, both locked by a party who harms himself as much as his counterparty.

**Front-running.** `claim` and `refund` pay fixed addresses, so seeing a share early lets no one redirect funds; seeing $a$ early helps only $M$, who is entitled to it. The one dangerous ordering is a share broadcast while its door is closed, which R2 and R3 rule out.

**Secret strength.** Shares are uniform in a 255-bit field. Recovering a share from its public point is the discrete logarithm problem that protects every Zcash and Bitcoin key; precomputed tables do not help against uniformly sampled keys.

**Key compromise.** An attacker who steals $b$ from $M$'s machine during a swap, together with $\mathsf{nk}$ and $\mathsf{rivk}$, could race $M$ for the note once $a$ is published. Because shares are per swap (R4), a compromise exposes one swap rather than a wallet, and $M$'s software sweeps within a block of the claim.

**Consensus validity.** The construction asks nothing of Zcash consensus beyond what FROST already uses: a valid full viewing key and a RedPallas signature under the rerandomized joint key. We confirmed this against the production verifier and a live node (Sections 10.1 and 10.2).

**Pool lifecycle.** The Orchard pool was sealed in 2026 but stayed spendable. A future upgrade could seal Ironwood the same way, so implementations should sweep promptly.

## 7. Privacy

| Observer | Learns | Does not learn |
|---|---|---|
| Zcash chain | Fully shielded transactions, their action counts and fees, and their times. | Any amount, any address, or any link between them or to Base. |
| Base chain | That maker $M$ escrowed $u$ USDC; a fresh payout address; $A$, $B$, and one share. | Anything on Zcash: none of these values appears there. |
| Maker | $v$, the funding transaction, and $T$'s payout and reply addresses. | $T$'s identity, balances or history. |
| Taker | Which transaction sweeps the joint note (she knows $\mathsf{nk}$). | Where the ZEC went: the sweep uses $\mathsf{ovk} = \perp$. |
| Light-wallet server | Client IP, which blocks it downloads, and which full transactions it fetches. | Note contents. |
| Relayer | $a$ shortly before it is published, and the payout address. | Anything on Zcash. |

**Table 2.** What each observer learns.

**The Base values do not appear on Zcash.** After settlement $A$, $B$, $\mathsf{ak} = A + B$ and one share are public on Base. On Zcash a spend publishes $\mathsf{rk} = \mathsf{ak} + [\alpha]\mathcal{G}$ for uniform $\alpha$, a uniform point independent of $\mathsf{ak}$, and an address depends on $\mathsf{ivk}$, hence on $\mathsf{nk}$ and $\mathsf{rivk}$, which are never published. An observer who knows $\mathsf{ak}$ can recognize neither the joint address nor its spend; in our tests no on-chain $\mathsf{rk}$ ever equalled $\mathsf{ak}$.

**The shared outgoing viewing key.** Anyone holding $\mathsf{FVK}_J$ can derive $\mathsf{ovk}_J$. If the sweep encrypted its output's recovery data under $\mathsf{ovk}_J$, the counterparty could read the destination and amount. Our tests recover the destination in exactly that case and fail to when the sweep uses $\mathsf{ovk} = \perp$, which is what our implementation does.

**The light-wallet server.** Compact blocks carry no memos, so our wallet fetches the full transaction for each note it decrypts, and the server learns which transactions concern us. A deployment should route light-wallet traffic over Tor, fetch decoy transactions, or download full blocks; our implementation does none of these yet.

**Timing and amounts.** An observer can correlate the time of `open` or `claim` with shielded transactions in the same interval; the anonymity set is every shielded transaction in that window. Either party can delay its Zcash transaction at no risk: $T$ can fund at any time before $t_0$, and $M$ can sweep at any time after the claim. ZEC amounts never appear on chain. The USDC amount is public by design, and standard lot sizes keep it from fingerprinting a trade.

**Comparison.** NEAR Intents, Maya and THORChain move ZEC through a transparent address and a custodian, so the deposit amount and time are public and the custodian can hold the output. Hash-locked swaps link both legs by the hash and use transparent ZEC. In Cloakroom the ZEC stays shielded throughout, no third party holds either asset, and timing is the only link between the chains.

## 8. The Dark Pool

Settlement removes the custodian; the request-for-quote layer removes the venue. A maker publishes a unified address, an EVM address and the pairs it quotes. Messages travel as memos on shielded outputs that may carry zero ZEC; a memo whose first byte is `0xFF` carries application data [37]. Every message begins with `0xFF`, `"CLKR"`, a version and a type.

| Message | Fields after the 7-byte header | Bytes |
|---|---|---|
| RFQ ($T \to M$) | rfq id (16), lots (1), $A$ (32), $\pi_A$ (64), commitment to $s_T$ (32), payout address (20), reply address (43) | 215 |
| Quote ($M \to T$) | rfq id (16), price (8), relayer fee (8), $B$ (32), $\pi_B$ (64), $s_M$ (32), escrow (20), maker EVM address (20), $t_0$ and $t_1$ offsets (8) | 215 |
| Accept ($T \to M$) | rfq id (16), $s_T$ (32), hash of the quote (32) | 87 |
| Fund ($T \to \mathsf{addr}_J$) | rfq id (16) | 23 |

**Table 3.** RFQ messages, as implemented. Each fits in one 512-byte memo.

A taker sends an RFQ to a maker with a fresh diversified reply address. The maker replies with a Quote to that address, and the taker answers with Accept, after which the maker calls `open` and settlement proceeds as in Section 4.4. A quote carries no signature: only the maker received the RFQ, so only the maker knows the reply address, and before funding the taker checks that the escrow was opened by the maker's EVM address with exactly the quoted parameters. A taker who asks several makers uses a different reply address for each. Lots have fixed sizes, so the price is the only trade-specific number on Base.

A maker learns what is asked (pair, side, size) and the taker's fresh addresses, which any quoting requires, but not who is asking. The chain sees only shielded transactions. A taker can send requests every period whether or not she trades, to hide when she is active, and requests can carry a small fee to deter spam.

## 9. Implementation

The implementation, written for this paper, has four parts:

- **Rust node** (`cloakroom`): key setup, proofs of knowledge, the message codec, maker and taker state machines, and a light wallet that builds v6 transactions through the PCZT roles. It signs joint spends externally with `apply_ironwood_signature`. It syncs either from a Zebra node over JSON-RPC or from a lightwalletd server over gRPC, starting from the server's Ironwood tree state at a birthday height. Built on `orchard` 0.16.0, `pczt` 0.10.0-pre.1 and `zcash_primitives` 0.31.0-pre.1.

- **Solidity**: the escrow and a Pallas library, 3,938 bytes of runtime code (Solidity 0.8.30, via-IR). The EVM side is driven through Foundry's `cast`.
- **Harness**: a script that starts a Zebra regtest node and a local EVM chain, deploys the contracts, funds two independent wallets, and runs each scenario with maker and taker as separate operating-system processes that share no state.
- **Tools**: `lwdscan` reports balances from compact blocks, and `txaudit` reports what a transaction reveals.

Both processes persist their state after every step. A party can be stopped at any stage and restarted, which is how the tests make parties "disappear".

## 10. Evaluation

### 10.1 Against the production verifier

A test program against `orchard` 0.16.0 with the post-NU6.3 circuit and Ironwood bundles checks:

1. The proofs of knowledge verify, a proof for one role fails for the other, and a rogue share $B = [b']\mathcal{G} - A$ fails.
2. `FullViewingKey::from_bytes` accepts $(A + B, \mathsf{nk}, \mathsf{rivk})$, and $\mathsf{ivk}_J$ decrypts a funding note and its memo.
3. A sweep signed with $b$ alone is rejected; one signed with $(a + b) + \alpha$ passes proof, spend-authorization and binding-signature verification, and so does the refund sweep.
4. With $\mathsf{ovk}_J$ the counterparty recovers the sweep's destination; with $\mathsf{ovk} = \perp$ it does not. No published $\mathsf{rk}$ equals $\mathsf{ak}$.

On a 10-core laptop the circuit keys took 1.8 s to build and a two-action proof took 0.61 s.

### 10.2 On a Zebra node

On Zebra 7.0.0-rc.0 in regtest with NU6.3 rules, a block reward was mined to a joint address. A v6 transaction built through the PCZT roles, with the real ZIP 244 sighash and an external signature under $(a+b)+\alpha$, was accepted to the mempool and mined (txid `6c7d4df5...a01616`, block 4). A copy differing only in its 64-byte spend-authorization signature, made with $b$ alone, was rejected (error $-25$). Zebra reports this as a proof failure because it verifies proofs and signatures in one batch; checked separately, the proof and binding signature of the copy were valid and only its spend signature failed.

### 10.3 Contract tests and gas

Fifteen Foundry tests cover the happy path, each row of Table 1, wrong shares at each door, window boundaries and off-curve points. A fuzz test of 5,000 random times, Ready states and call orders never paid both sides and always conserved the token supply. The Pallas library reproduces the Rust vectors for $\mathcal{G}$, $A$, $B$ and $[q-1]\mathcal{G} = -\mathcal{G}$.

| Operation | Foundry, mock token | Base Sepolia, Circle USDC |
|---|---|---|
| Pallas check $[k]\mathcal{G} \stackrel{?}{=} (x, y)$ | 467,341 | — |
| `open` | 279,042 | 297,756 |
| `setReady` | 32,440 | 32,440 |
| `claim` | 553,364 | 564,054 |
| `refund` | 528,531 | 505,221 |

**Table 4.** Gas used.

A swap costs about $9 \times 10^5$ gas, a few cents on Base. An earlier Solidity Pasta library reports 389,487 gas for a projective scalar multiplication [38]. A table of precomputed multiples of $\mathcal{G}$ would cut our cost further; we have not needed it.

### 10.4 Two-party scenarios

Table 5 lists the seven scenarios, all run in one session on fresh chains, with 43 checks including balances exact to the zatoshi. Scenarios s2, s3, s4 and s6 needed the share negation of Section 4.2 and s1 and s7 did not, so both branches ran on chain.

| | Scenario | Outcome |
|---|---|---|
| s1 | Happy path: RFQ, quote, accept, open, fund, ready, claim, sweep | settled |
| s2 | Taker accepts, then disappears before funding | refunded before $t_0$ |
| s3 | Taker funds and disappears; returns after $t_1 - \Delta$ | R2 refuses the claim; refunded; ZEC recovered |
| s4 | Maker opens, then disappears without setting Ready | claimed after $t_0$; maker sweeps on return |
| s5 | Maker sends a rogue key $B = [b']\mathcal{G} - A$ | quote rejected; nothing funded |
| s6 | Taker underfunds the joint address | no Ready; refunded; ZEC recovered |
| s7 | Happy path with NU7 active | settled |

**Table 5.** Scenarios on Zebra regtest and a local EVM chain. All passed.

In every scenario that moves ZEC, the taker lost exactly the expected fees. In s1, s3 and s7 we read every Zcash transaction either party made back from the node: all were v6 with two Ironwood actions, no transparent inputs or outputs, and an Ironwood value balance equal to the 10,000-zatoshi fee.

The first runs also found a liveness gap. With $t_0$ only 40 seconds after `open`, the taker waited for the safe head (which lags about 32 blocks on the local chain) and funded just as the maker's no-funding refund fired. The protocol resolved it safely: the maker refunded, and the taker read $b$ and recovered her note. We added rule R1b so that a taker never funds that close to $t_0$.

### 10.5 Public networks

We ran two swaps between Zcash testnet, which runs NU7 rules, and Base Sepolia. The Zcash wallets synced through the public `testnet.zec.rocks` lightwalletd server; the escrow is deployed at `0x2df30788...39a1` with its source verified on Blockscout. Lots were 0.01 TAZ for 1 USDC, with a 0.1 USDC relayer fee, $t_0$ = open+15 minutes, $t_1 = t_0 + 15$ minutes, and $k_Z = 3$.

| Swap | Result, checked on chain |
|---|---|
| 1, happy path | RFQ, quote and accept memos in blocks 4,501,217–225; funding in 4,501,234; `setReady` after 4 confirmations; claim by the relayer; maker's joint-key sweep in 4,501,240. The fresh payout address received 0.9 USDC, the relayer 0.1 USDC, and the maker 0.01 TAZ. |
| 2, refund | The taker funded in block 4,501,274 and its process exited. The maker set Ready, waited past $t_1$, and refunded, publishing $b$. The returning taker swept the note back with $a + b$ in block 4,501,358, losing exactly four 0.0001 TAZ fees. |

**Table 6.** Swaps between Zcash testnet and Base Sepolia, October 14, 2026.

All eleven Zcash transactions involved (a transfer to fund the maker and five per swap) are v6, with two Ironwood actions, no transparent inputs or outputs, no Sapling or Orchard-pool activity, and an Ironwood value balance equal to the fee.

The public networks exposed four problems that local chains did not, all now fixed:

- The public RPC served a stale nonce, so the maker's `open` collided with its `approve`. Sends now retry, and `open` checks the escrow first, so a retry is idempotent.
- The public RPC limits log queries to 200 blocks, so lookups now query in chunks.
- A network error stopped a process. Errors are now logged and retried; the maker rode out two RPC timeouts in swap 2.

- A balance read just after a claim returned stale data. Balances are now checked separately.

After these fixes, all seven local scenarios still pass.

## 11. Limitations

**Liveness.** Takers must claim within a window and makers must respond before $t_0$. **Liquidity.** Atomic swaps have historically traded small volumes; the RFQ layer lets professional makers provide depth but does not create them. **Our implementation** supports one maker per request, trusts the relayer for liveness only, does not use Tor, and lets the light-wallet server see which transactions it fetches. Its contracts are unaudited. **Not tested:** forced inclusion through Ethereum, Base mainnet, Zcash mainnet, and the production window lengths of Section 5. **Quantum recovery** is unavailable for joint notes (Section 4.2). **Linkability by timing** remains (Section 7). **Chains without elliptic-curve arithmetic**, such as Bitcoin, would need adaptor signatures and a cross-group proof [18, 21] in place of our escrow.

## 12. Conclusion

We have proposed a way to trade shielded Zcash without leaving the shielded pool and without a custodian. The spending key of a fresh Ironwood address is split between the two parties, and the counterparty's tokens are held by a contract whose two exits each publish one share. Because taking either exit completes the other party's knowledge of the key, the swap settles atomically or unwinds atomically. The Zcash leg is ordinary shielded transactions, the EVM leg reveals the price, and the request-for-quote messages ride inside encrypted memos. The construction needs no change to Zcash consensus, and it already runs between Zcash testnet and Base Sepolia.

## References


[1] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, M. Virza, "Zerocash: Decentralized anonymous payments from Bitcoin," *IEEE S&P*, 2014.

[2] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox, "Zcash Protocol Specification," version 2026.7.0, `https://zips.z.cash/protocol/protocol.pdf`.

[3] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[4] D. Hopwood, "ZIP 258: Deployment of the NU6.3 network upgrade," `https://zips.z.cash/zip-0258`.

[5] D. Hopwood, "ZIP 326: NU6.3 consequences for wallets," `https://zips.z.cash/zip-0326`.

[6] D. Hopwood, J. Grigg, "ZIP 2005: Ironwood quantum recoverability," `https://zips.z.cash/zip-2005`.

[7] D. Ojha, "ZIP 259: Deployment of the NU7 network upgrade," `https://zips.z.cash/zip-0259`.

[8] G. Kappos, H. Yousaf, M. Maller, S. Meiklejohn, "An empirical analysis of anonymity in Zcash," *USENIX Security*, 2018.

[9] J. Quesnelle, "On the linkability of Zcash transactions," arXiv:1712.01210, 2017.

[10] A. Biryukov, D. Feher, "Privacy and linkability of mining in Zcash," *IEEE CNS*, 2019.

[11] K. Huseynov, A. Shahzaib, I. A. Seres, J. Tapolcai, "A tattered cloak of invisibility: Measuring anonymity loss in Railgun on Ethereum," arXiv:2606.25926, 2026.

[12] NEAR Intents, "Supported chains," `https://docs.near-intents.org/resources/chain-support`.

[13] C. Monasterio, "$589K USDT stuck for 50 days after Zodl swap," Crypto Economy, Sep. 11, 2026.

[14] CoinCentral, "NEAR Intents hacked for $3.8 million," Oct. 2, 2026.

[15] T. Nolan, "Alt chains and atomic transfers," bitcointalk.org, 2013.

[16] M. Herlihy, "Atomic cross-chain swaps," *PODC*, 2018.

[17] J. Graber, "ZIP 300: Cross-chain atomic transactions," `https://zips.z.cash/zip-0300`, 2017.

[18] J. Gugger, "Bitcoin–Monero cross-chain atomic swap," IACR ePrint 2020/1126.

[19] COMIT, "xmr-btc-swap," https://github.com/comit-network/xmr-btc-swap; continued as eigenwallet.

[20] E. Binks, "Cross-chain atomic swaps between Ethereum and Monero," AthanorLabs, 2022.

[21] S. Noether, "Discrete logarithm equality across groups," MRL-0010, 2018.

[22] S. A. K. Thyagarajan, G. Malavolta, P. Moreno-Sanchez, "Universal atomic swaps: Secure exchange of coins across all blockchains," *IEEE S&P*, 2022.

[23] F. Engelmann, T. Kerber, M. Kohlweiss, M. Volkhov, "Zswap: zk-SNARK based non-interactive multi-asset swaps," *PoPETs* 2022(4).

[24] Atheon, "Zwap: Trustless shielded atomic swaps for Zcash (early access)," Zcash Community Forum, May 2026.

[25] Stallion Labs, "Grant application: zswap," Zcash Community Grants issue #443, 2026.

[26] Renegade, documentation, https://docs.renegade.fi.

[27] Penumbra, "Decentralized exchange," https://protocol.penumbra.zone/main/dex.html.

[28] K. Nuttycombe, D. Hopwood, J. Grigg, "ZIP 244: Transaction identifier non-malleability," https://zips.z.cash/zip-0244.

[29] S. Bowe, J. Grigg, D. Hopwood, "Recursive proof composition without a trusted setup," IACR ePrint 2019/1021.

[30] J. Grigg, "ZIP 374: Partially created Zcash transaction format," https://zips.z.cash/zip-0374.

[31] C. Komlo, I. Goldberg, "FROST: Flexible round-optimized Schnorr threshold signatures," *SAC*, 2020.

[32] C. Gouvêa, C. Komlo, D. Connolly, "ZIP 312: FROST for spend authorization multisignatures," https://zips.z.cash/zip-0312.

[33] G. Maxwell, A. Poelstra, Y. Seurin, P. Wuille, "Simple Schnorr multi-signatures with applications to Bitcoin," *Designs, Codes and Cryptography*, 2019.

[34] D. Hopwood, K. Nuttycombe, J. Grigg, "ZIP 317: Proportional transfer fee mechanism," https://zips.z.cash/zip-0317.

[35] Base, "Reliability and liveness," https://docs.base.org/specifications/security/reliability-and-liveness.

[36] Optimism, "Forced transaction," https://docs.optimism.io/stack/transactions/forced-transaction.

[37] J. Graber, J. Grigg, "ZIP 302: Standardized memo field format," https://zips.z.cash/zip-0302.

[38] Z. Zhang, "pasta-solidity," https://github.com/zhenfeizhang/pasta-solidity.

## Appendix: Test Vectors

$\mathcal{G}$, as used in the escrow and extracted from `orchard` 0.16.0 (its compressed encoding matches the `reddsa` crate's basepoint constant):

```
x = 0x375523b328f1d6063b8d187c3e5f445f0c7f0ce37b70a10c8d1a7284b875c963
y = 0x1ad0357fdf1a66db7b10bcfcfed624fbdfc914fec005bdd84ce33e817b0c3bc9
```

A share and its point after sign normalization:

```
a   = 0x195661e2e4789b7f57137b2e777736d79720270638d12e4882fd95e52a73d6b3
A.x = 0x17a182b1eb9594581ecaad0fcb61546f4c366cc57b9575ddde22ca56963c304c
A.y = 0x19b252972f768cc5708b251baca77a9afd02dddd9531510aafb13c4824d7b61f
```